

Arquitetura de Software

Padrões de criação

Adriano Rivolli

rivolli@utfpr.edu.br

Universidade Tecnológica Federal do Paraná (UTFPR)
Câmpus Cornélio Procópio
Departamento de Computação

Conteúdo

- 
- 1** Singleton
 - 2** Prototype
 - 3** Builder
 - 4** Factory Method
 - 5** Abstract Factory

Singleton

Definição

Singleton

Garante que uma classe tenha apenas uma instância, enquanto provê um ponto de acesso global para essa instância.

<https://refactoring.guru/pt-br/design-patterns/singleton>

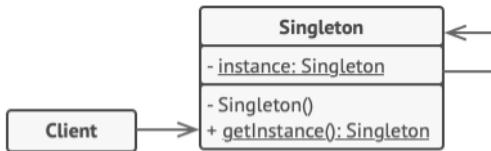
Problema

- Garante que uma classe tenha **uma única** instância
 - ▶ Controlar o acesso a algum recurso compartilhado
- Fornece um ponto de **acesso global** para a instância
 - ▶ Eliminação de variáveis globais
 - ▶ Protege a instância de ser sobrescrita

Solução

- 
- 1 Fazer o construtor padrão privado
 - 2 Criar um método estático de criação que age como um construtor
 - ▶ Chama o construtor privado
 - ▶ Salva o objeto criado em um campo estático
 - ▶ Todas as chamadas seguintes para esse método retornam o objeto em cache

Solução



- 1 A classe **Singleton** declara o método estático `getInstance` que retorna a mesma instância de sua própria classe.

O construtor da singleton deve ser escondido do código cliente. Chamando o método `getInstance` deve ser o único modo de obter o objeto singleton.

```

if (instance == null) {
    // Atenção: se você está criando uma
    // aplicação com apoio multithreading,
    // você deve colocar um thread lock aqui.
    instance = new Singleton()
}
return instance
    
```

Implementação

TS singleton.ts > ...

```
1  class Singleton {  
2      private static instance: Singleton;  
3  
4      private constructor() { }  
5  
6      public static getInstance(): Singleton {  
7          if (!Singleton.instance) {  
8              Singleton.instance = new Singleton();  
9          }  
10  
11         return Singleton.instance;  
12     }  
13  
14     public fazAlgunaCoisa() {  
15         // TODO ...  
16     }  
17 }
```


Aplicações

- Conexão com banco de dados
- Gerenciador de configurações
- Registro de logs
- Controle de estados

Prós

- Garantia que uma classe só terá uma única instância
- Ponto de acesso global para aquela instância
- O objeto singleton é inicializado somente quando for pedido pela primeira vez

Contras

- 
- Viola o princípio de responsabilidade única. O padrão resolve dois problemas de uma só vez
 - O padrão requer tratamento especial em um ambiente *multithreaded*
 - Pode ser difícil realizar testes unitários do código cliente do Singleton

Prototype

Definição

Prototype

Permite copiar objetos existentes sem fazer seu código ficar dependente de suas classes.

<https://refactoring.guru/pt-br/design-patterns/prototype>

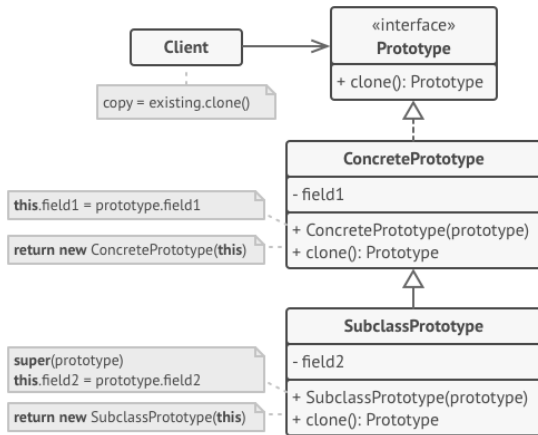
Problema

- Como fazer para criar uma cópia exata de um objeto?
 - ▶ Criar um novo objeto da mesma classe
 - ▶ Copiar os valores de todos os atributos para o novo objeto
 - ▶ *Alguns campos do objeto podem ser privados*
 - ▶ *É necessário saber a classe do objeto*
- O código se torna dependente daquela classe
 - ▶ Algumas vezes você só sabe a interface que o objeto segue

Solução

- Delegar o processo de clonagem para o próprio objeto que está sendo clonado
- O padrão declara um interface comum para todos os objetos que suportam clonagem
 - ▶ `clone()`
- Um objeto que suporta clonagem é chamado de um protótipo
- Alternativa a subclasses

Solução



Implementação

TS prototype.ts > ...

```

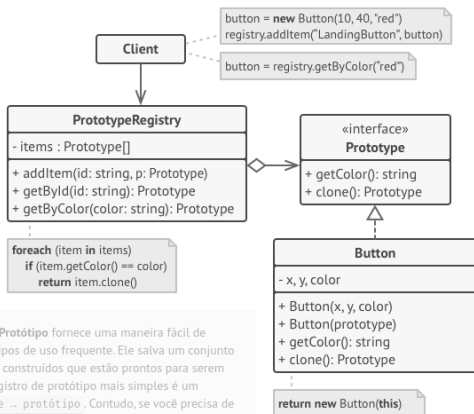
1  interface Prototype<T> {
2      clone(): T;
3  }
4
5  class MeuPrototipo implements Prototype<MeuPrototipo> {
6      public primitive: any;
7      public component: object;
8      public circularReference: ComponentWithBackReference;
9
10     public constructor(primitive: any, component: object) {
11         this.primitive = primitive;
12         this.component = component;
13         this.circularReference = new ComponentWithBackReference(this);
14     }
15
16     public clone(): this {
17         const clone = Object.create(this);
18         clone.component = Object.create(this.component);
19         clone.circularReference = new ComponentWithBackReference(clone);
20         return clone;
21     }
22 }

```

Gerenciador de Protótipos

- 
- Fornece uma maneira fácil de acessar protótipos de uso frequente
 - Salva um conjunto de objetos pré-construídos que estão prontos para serem copiados
 - Pode ser implementado com um *hashmap*

Gerenciador de Protótipos



- 1 O Registro do Protótipo fornece uma maneira fácil de acessar protótipos de uso frequente. Ele salva um conjunto de objetos pré construídos que estão prontos para serem copiados. O registro de protótipo mais simples é um hashmap nome → protótipo. Contudo, se você precisa de um melhor critério de busca que apenas um nome simples, você pode construir uma versão muito mais robusta do registro.

Aplicações

- 
- Criação de objetos com configurações padrão
 - Sistemas de jogos (clonar personagens, armas, itens)
 - Sistemas de formulários dinâmicos
 - Evitar a construção de uma hierarquia de classes
 - Quando uma classe possui poucos estados possíveis

Prós

- É possível clonar objetos sem os acoplar a suas classes concretas
- Se livrar de códigos de inicialização repetidos em troca de clonar protótipos pré-construídos
- Produzir objetos complexos mais convenientemente
- Uma alternativa para herança quando lidar com configurações pré determinadas para objetos complexos

Contras

- Clonar objetos complexos que têm referências circulares pode ser bem complicado

Desafio proposto (atividade)

- 1 Implemente uma interface chamada **Prototype** com o método abstrato **clone()**
- 2 Implemente uma classe com ao menos 3 atributos privados que siga a interface **Prototype**
- 3 Implemente um gerenciador de protótipos que possui uma única instância chamado de **PrototypeManager** com os seguintes métodos:
 - ▶ **register(key, prototype)** para registrar um protótipo com uma chave
 - ▶ **get(key)** que retorna um clone do protótipo associado à chave
 - ▶ **getInstance()** método estático que retorna o objeto **PrototypeManager**
- 4 Instancie um conjunto de objetos e registre-os no gerenciador de protótipos

Builder

Definição

Builder

Permite a construção de objetos complexos passo a passo. O padrão permite que se produzam diferentes tipos e representações de um objeto usando o mesmo código de construção.

<https://refactoring.guru/pt-br/design-patterns/builder>

Problema

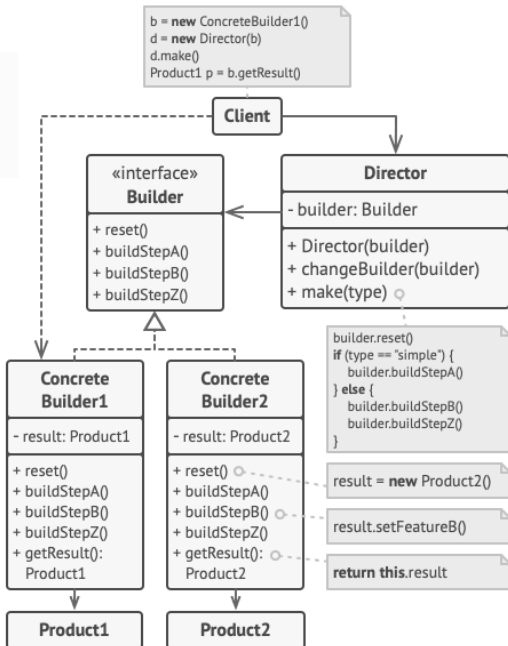
- Objetos complexos podem necessitar de uma inicialização passo a passo trabalhosa de muitos campos e objetos agrupados
 - ▶ Construtor com muitos parâmetros
 - ▶ Inicialização fica espalhada por todo o código cliente
 - ▶ Uso de parâmetros nulos/vazios
- Hierarquia de classes complexas
 - ▶ Causado pelo acréscimo de novos parâmetros

Solução

- Extração do código de construção do objeto para fora da sua própria classe
- Uma classe **Builder** é utilizada para a criação das partes do novo objeto
 - ▶ A construção é organizada em etapas
 - ▶ As etapas podem ser opcionais
 - ▶ As etapas podem ser alternativas
- Um objeto resultante só poderá ser acessado quando finalizada a construção

Solução

- Uma classe **Diretor** pode definir as etapas e usar diferentes **builders**
 - ▶ Seu papel consiste em definir a ordem em que as etapas de construção serão executadas
 - ▶ Seu uso é opcional, pois é possível chamar as etapas de construção usando diretamente os **builders**
 - ▶ Vantagem: esconde os detalhes da construção
- O **diretor** define a ordem e o **builder** provê a implementação das etapas



Implementação

- 1 Definir uma interface **Builder** para definir as etapas comuns de construção
- 2 Crie uma classe **builder** concreta para cada **representação** do produto e implemente suas etapas de construção
 - ▶ Se **todos** os objetos gerados possuem uma interface comum, o método de sua obtenção é declarada na interface
- 3 Possivelmente, criar uma classe **Diretor**
- 4 A aplicação cria tanto o **builder** quanto o **diretor**

Encadeamento de métodos

- *Method chaining*
- Os métodos `set/build` retornam o próprio objeto (`this`)
- Permite que chamadas sejam feitas em sequência
 - ▶ `obj.setA().setB().setC()`
 - ▶ Código mais legível e fluido
- Exemplo:
 - ▶

```
setNome(nome){  
    this.nome = nome;  
    return this;  
}
```

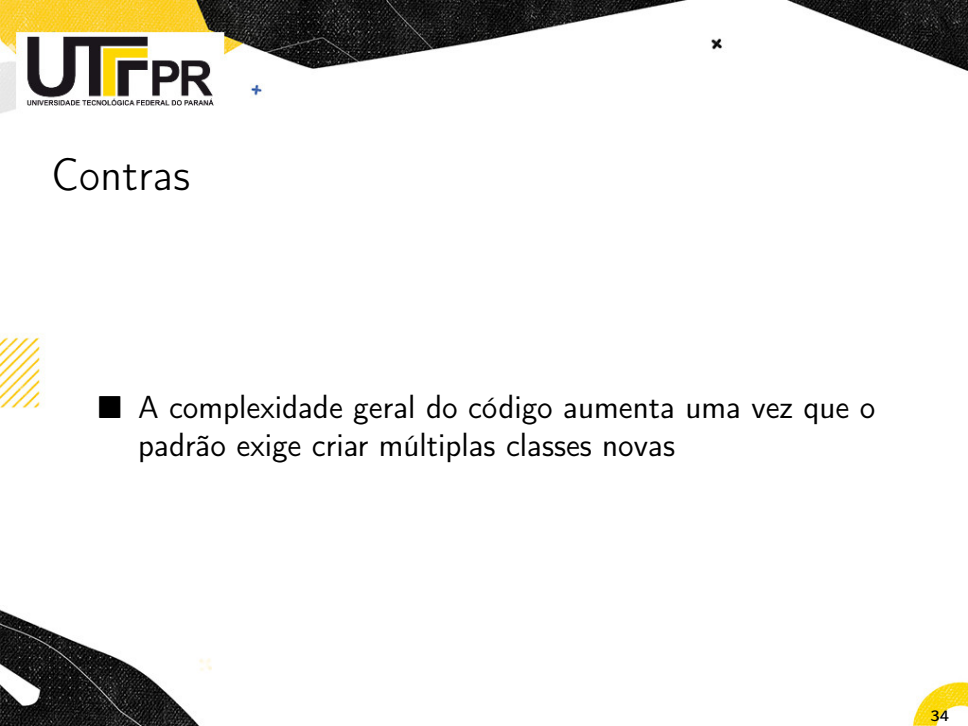
Aplicações

- 
- Criação de objetos com muitos parâmetros
 - Construção de componentes de interface (UI)
 - Cenários com muitos construtores sobrecarregados
 - Criar diferentes representações do mesmo produto/processo
 - Quando há etapas similares de construção, porém com pequenas diferenças

Prós

- É possível construir objetos passo a passo, adiar as etapas de construção ou rodar etapas recursivamente
- Reutilizar o mesmo código de construção quando construindo várias representações de produtos
- Princípio de responsabilidade única. é possível isolar um código de construção complexo da lógica de negócio do produto

Contras

- 
- A complexidade geral do código aumenta uma vez que o padrão exige criar múltiplas classes novas

Factory Method

Definição

Método Fábrica

Fornece uma interface para criar objetos em uma superclasse, mas permite que as subclasses alterem o tipo de objetos que serão criados.

<https://refactoring.guru/pt-br/design-patterns/factory-method>

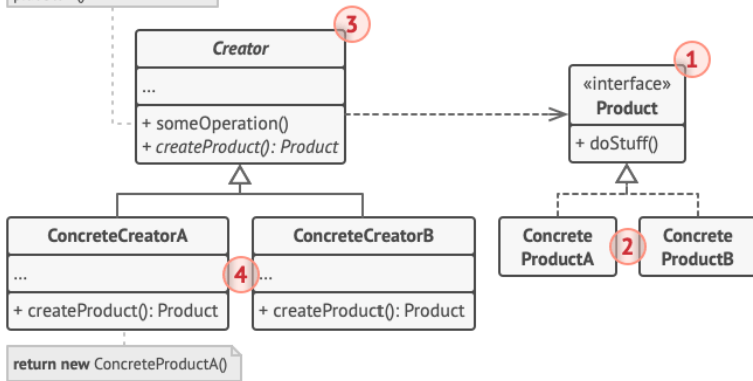
Problema

- Quando uma classe precisa criar objetos de outras classes, isso gera acoplamento
- O código cliente (que usa os objetos)
precisa conhecer diretamente as classes concretas

Solução

- Substituir chamadas diretas de construção de objetos (usando o operador `new`) por chamadas para um método fábrica especial
 - ▶ Evita acoplamento direto do cliente com classes concretas
 - ▶ Delega a criação dos objetos para subclasses ou métodos especializados
 - ▶ Permite extensão: novas variantes de produtos podem ser introduzidas sem mudar o código cliente

Product p = createProduct()
p.doStuff()



Implementação

- 
- 1 O **Produto** declara a interface, que é comum a todos os objetos que podem ser produzidos pelo criador e suas subclasses
 - 2 **Produtos Concretos** são implementações diferentes da interface do produto
 - 3 A classe **Criador** declara o método fábrica que retorna novos objetos produto
 - 4 **Criadores Concretos** sobrescrevem o método fábrica base para retornar um tipo diferente de produto
- 

Aplicações

- Drivers de banco de dados
- Frameworks de Interface Gráfica
- Parser de documentos
- APIs de pagamento
- Cenários:
 - ▶ Quando desejar fornecer em uma biblioteca ou framework uma maneira de estender seus componentes internos
 - ▶ Quando não souber de antemão os tipos e dependências exatas dos objetos
 - ▶ Quando desejar reutilizar objetos existentes em vez de recriá-los sempre

Prós

- 
- Evita acoplamentos firmes entre o criador e os produtos concretos
 - Você pode mover o código de criação do produto para um único local do programa
 - Adicionar novos tipos de produtos no programa sem quebrar o código existente

Contras

- O código se torna mais complicado, pois é necessário introduzir subclasses novas para implementar o padrão

Abstract Factory

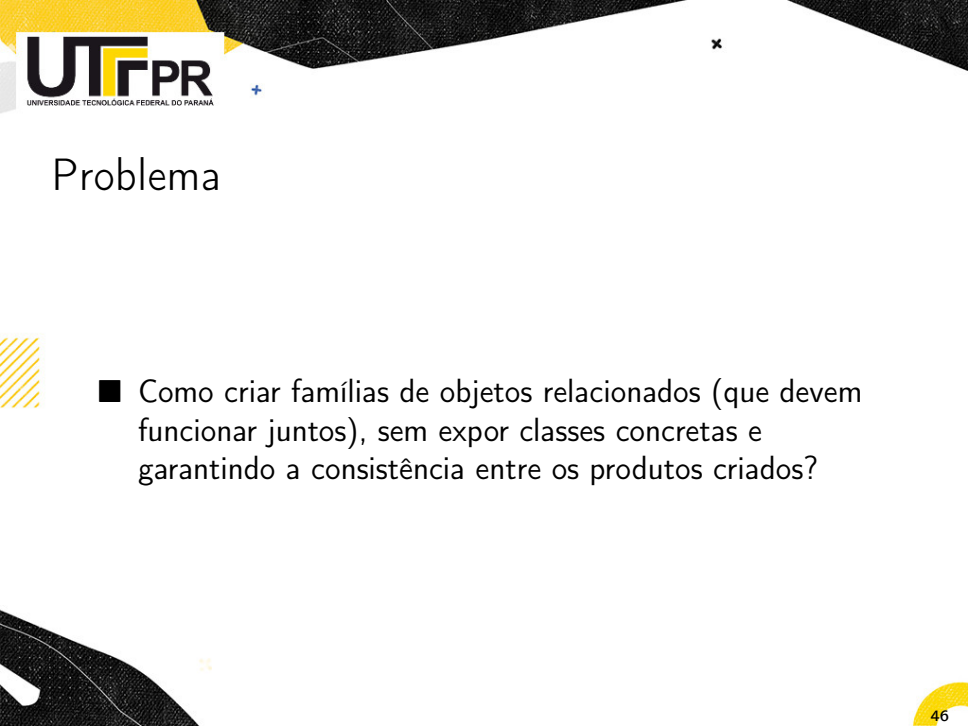
Definição

Fábrica Abstrata

Permite que se produza famílias de objetos relacionados sem ter que especificar suas classes concretas.

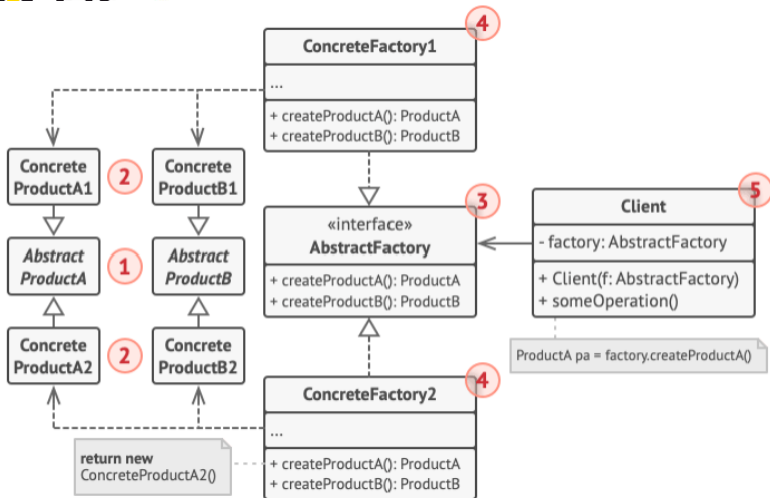
<https://refactoring.guru/pt-br/design-patterns/abstract-factory>

Problema

- 
- Como criar famílias de objetos relacionados (que devem funcionar juntos), sem expor classes concretas e garantindo a consistência entre os produtos criados?

Solução

- Declarar explicitamente interfaces para cada produto distinto da família de produtos
- Declarar a fábrica abstrata—uma interface com uma lista de métodos de criação para todos os produtos que fazem parte da família de produtos
- Para cada variante de uma família de produtos é criado uma classe fábrica separada



Implementação

- 
- 1 **Produtos Abstratos** declaram interfaces para um conjunto de produtos distintos da mesma família
 - 2 **Produtos Concretos** são várias implementações de produtos abstratos, agrupados por variantes
 - 3 A interface **Fábrica Abstrata** declara um conjunto de métodos para criação de cada um dos produtos abstratos
 - 4 Cada **fábrica concreta** corresponde a uma variante específica de produtos e cria apenas aquelas variantes de produto
- 

Aplicações

- Temas de Interface
- Sistemas com diferentes regras de localidade
 - ▶ Trabalhar com diversas famílias de produtos relacionados

Prós

- Certeza que os produtos que você obtém de uma fábrica são compatíveis entre si
- Evitar um vínculo forte entre produtos concretos e o código cliente
- Extrair o código de criação do produto para um lugar
- Adicionar novas variantes de produtos no programa sem quebrar o código existente

Contras

- O código se torna mais complicado, pois é necessário introduzir subclasses novas para implementar o padrão